

Table of contents

Abstract.....	4
Introduction	5
Objectives of this project	6
Project	6
Features	6
Plan	7
Interface Design and User Experience	7
Designs	10
Rebeca Purple.....	13
Architecture.....	13
Frontend Application.....	13
State Management.....	14
Router	14
Pages	15
Backend Application	15
Data storage (database)	15
Routes (endpoints).....	15
Architecture Overview	15
Implementation	17
Results and Reflection.....	19
Release	19
Reflection	19
Future work	19
References	20
Credit	20
Tools	20
Figures	20
Tables	21
Bibliography	21

Abstract

This project introduces a cutting-edge web platform that harnesses the power of AI to transform the way we create and share stories. Traditional publishing workflows are often cumbersome, and this project aims to address this by offering a suite of AI-powered tools. By leveraging advanced technologies like Vertex-AI, Bing Image Search API, and Grammarly API, the web application streamlines content creation, enhances originality, and elevates user experience. The application was entirely built with Javascript, React and NodeJS, boasting an agile architecture. The platform has been refined to achieve efficient content generation, visually stunning stories, and a rich content library. Ultimately, the application aspires to streamline publishing, enrich storytelling, and foster a thriving ecosystem for creators and consumers.

Introduction

The traditional publishing process can be challenging, filled with time-consuming hurdles that stifle creativity. The journey of a story from an author's initial idea to a reader's hands involves a network of stakeholders, each playing a vital role in a book's success (Figure 1).

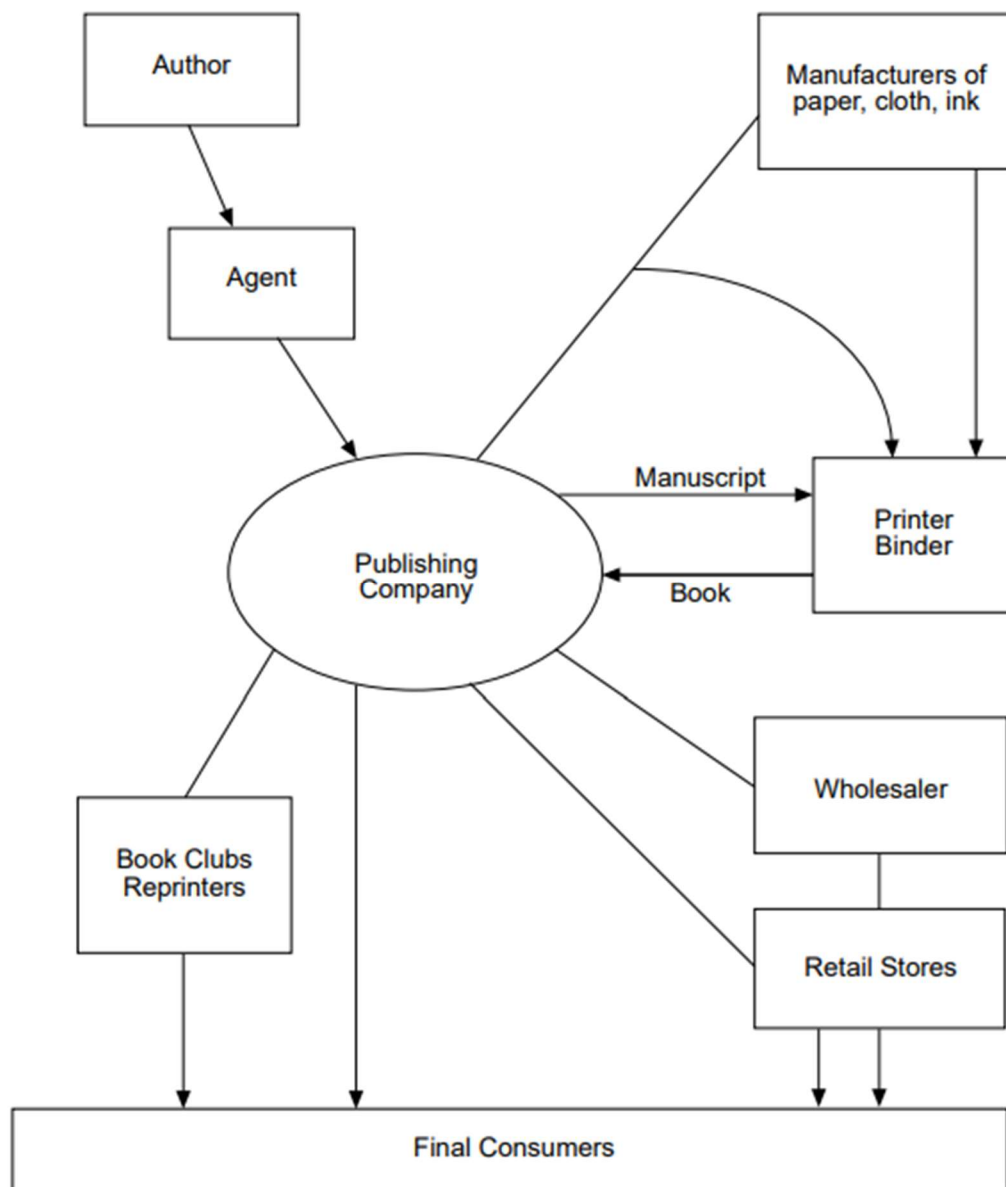


Figure 1 - Structure of the book publishing industry (Vanier, 1973 p.5)

The author starts the process by transforming an idea into a written work, often requiring extensive research and writing periods. On finishing, the author may partner with a literary agent to find the perfect publishing house for the manuscript. After successful pairing, the publisher

starts the editing process before the manuscript is sent to a printer. The printer transforms it into a physical book. The finished product then travels through distribution channels, involving wholesalers and retailers, before reaching bookstores and readers.

While this established system functions, it can be lengthy and challenging for aspiring authors. This project seeks to address these challenges and introduce a new approach to story publishing. The main goal for this project is outlined in the following objectives:

Objectives of this project

- Develop an intuitive editor where users can input text, and generate images based on prompts.
- Implement a robust plagiarism detection system to ensure the originality of user-generated content.
- Enable users to seamlessly create, edit, and share their stories.

Project

This project was started with Agile in mind. A backlog of tasks that were required for the web application was created. Next, the features to be built were defined and tasks prioritized accordingly. After each development cycle, we reviewed the working prototype, evaluated the project plan and tasks and made adjustments as needed.

Features

The below features were identified:

Table 1 - Application features

Story editor	craft story with a user-friendly interface
Story reader	immersive and captivating reading experience
Text generator	spark creativity with AI-powered text suggestions and story continuations
Image generator	generates images that perfectly capture scenes
Story styler	customize the look and feel of your story
Plagiarism detector	flags plagiarism, edits text to remove it

User authenticator	secures application allowing and privacy control
Published book library	shows a vast collection of published stories
My book library	manage your personal collection of stories, keeping track of your progress and drafts

Plan

To ensure smooth development and prioritize features effectively, we utilized Github as my project management tool. This platform allowed me to create a roadmap for StoryNasi's iterations and maintain project organization, and also provided crucial insights for efficient task completion. Each feature was assigned a dedicated card within Github, packed with details like, status, size (complexity), description, iteration, priority level, and estimated completion dates. With this information readily available, I could identify tasks requiring more time and strategically plan a workflow to deliver all required functionality on schedule.

Interface Design and User Experience

My initial approach was to outline the user flow with a flowchart in Figma (Figure 2). This visual representation helped us understand the overall process users would experience when interacting with the application.

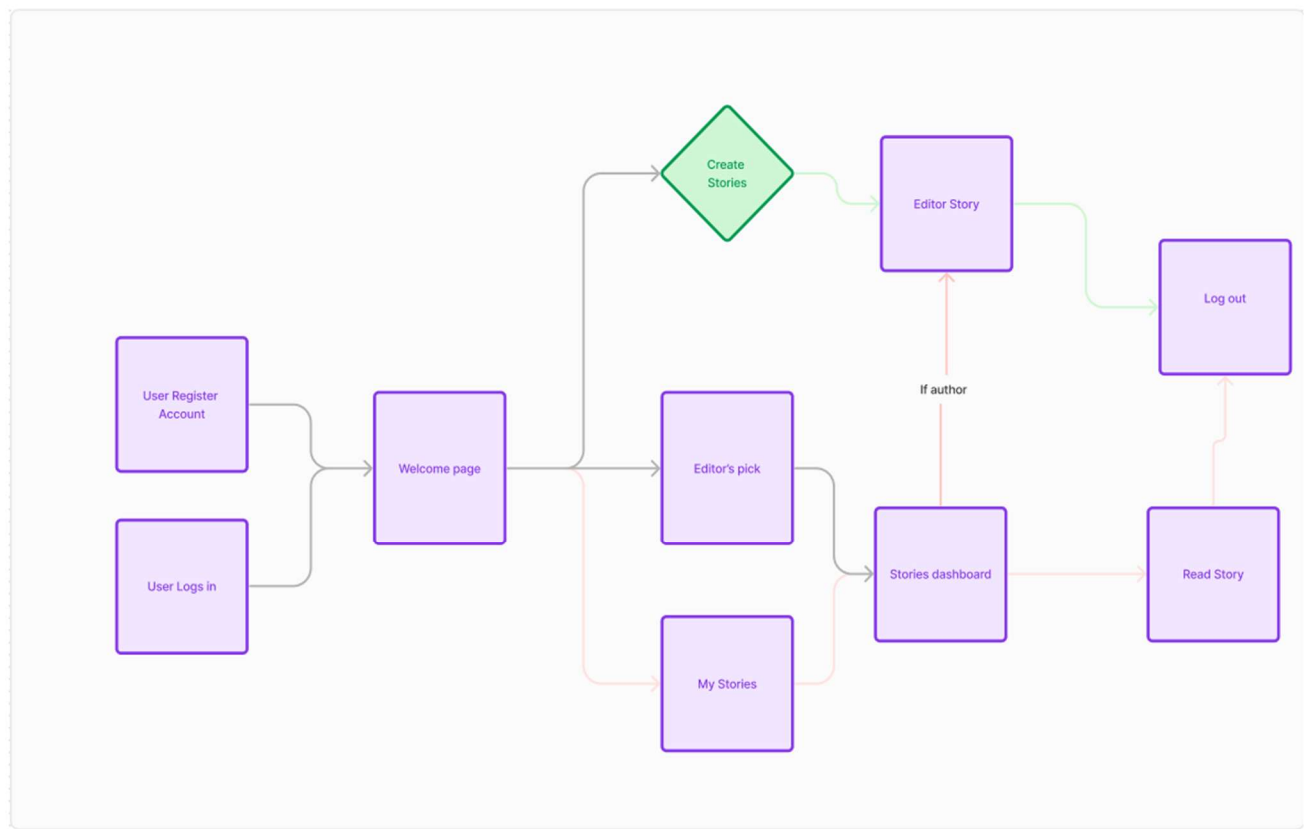


Figure 2 - User flow diagram

After defining the user journey, we focused on designing the backend database schema.

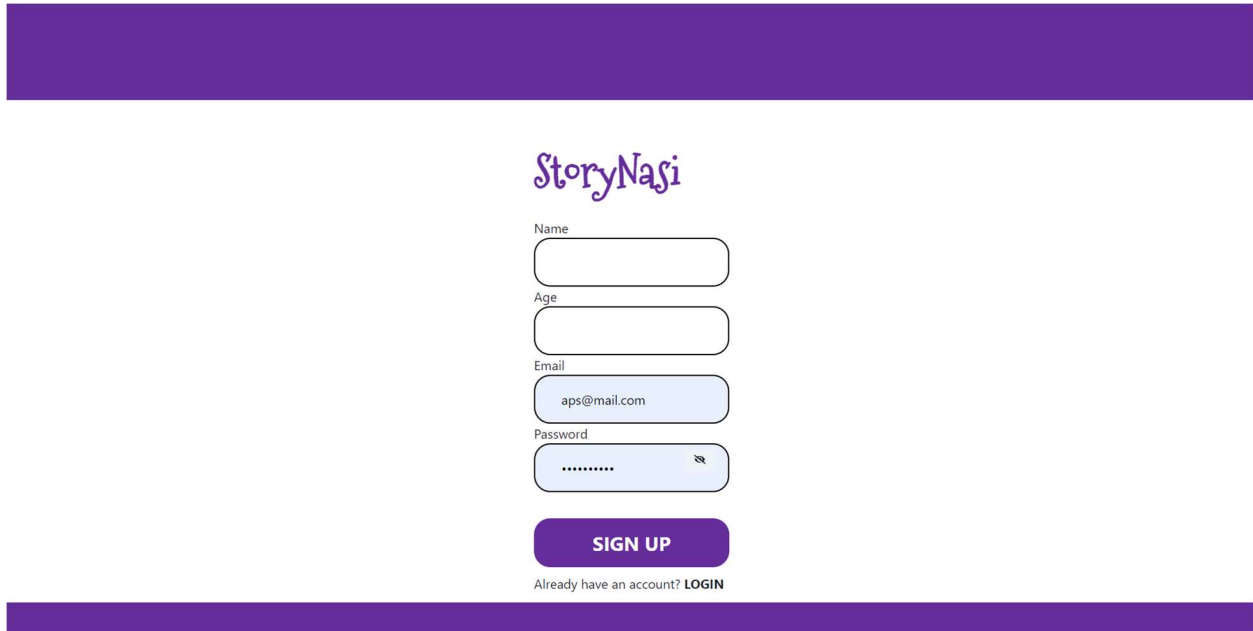


Figure 3 - Database schema

The next step was to design the individual pages and core components that would make up the application's interface.

Designs

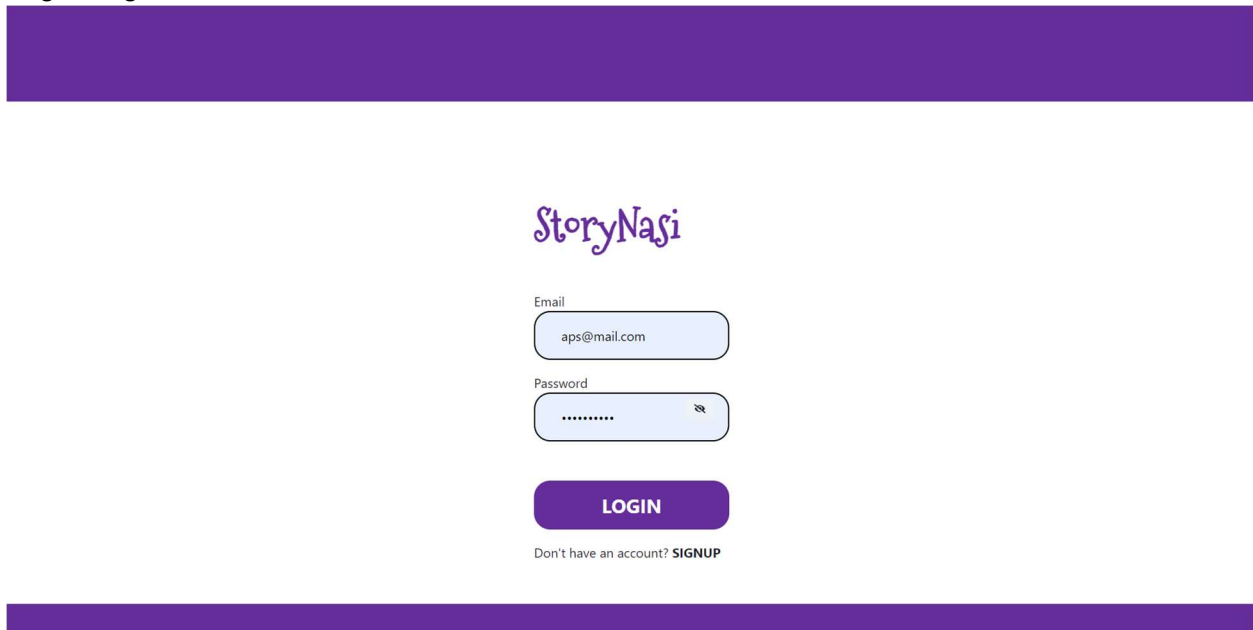
Signup page



The Signup page design features a purple header bar at the top. Below it, the StoryNasi logo is centered. The form consists of four input fields: Name, Age, Email (pre-filled with 'aps@mail.com'), and Password (masked with dots and a toggle icon). A purple 'SIGN UP' button is positioned below the fields. At the bottom, a link reads 'Already have an account? LOGIN'.

Figure 4 - Signup page

Login Page



The Login page design features a purple header bar at the top. Below it, the StoryNasi logo is centered. The form consists of two input fields: Email (pre-filled with 'aps@mail.com') and Password (masked with dots and a toggle icon). A purple 'LOGIN' button is positioned below the fields. At the bottom, a link reads 'Don't have an account? SIGNUP'.

Figure 5 - Login page

Home Page

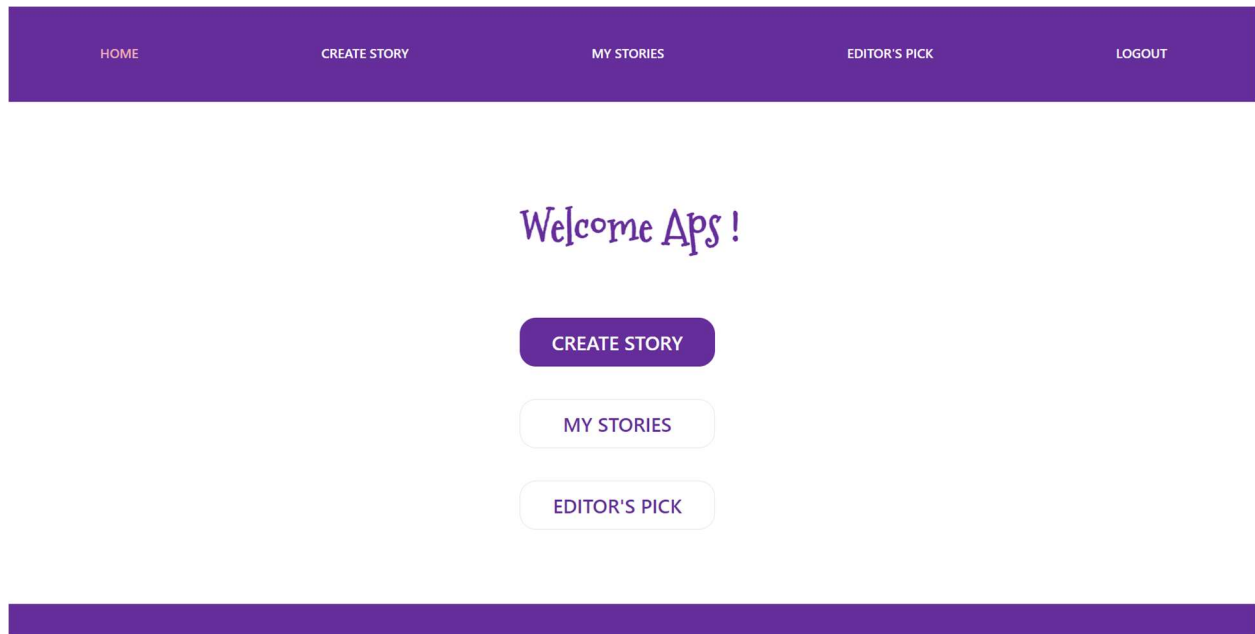


Figure 6 - Home page

Create Story Page

The screenshot shows the "Create Story" page. It features a purple navigation bar at the top with the same five links as the Home page. Below the navigation bar, the page contains three input fields: a "Title" field, a "Description" field, and an "ImageURL" field. The "ImageURL" field has a placeholder text: "Type image url or a prompt to generate an image". Below these fields is a purple button labeled "Start My Story". The page has a light gray background.

Figure 7 - Create story page

Writing Page

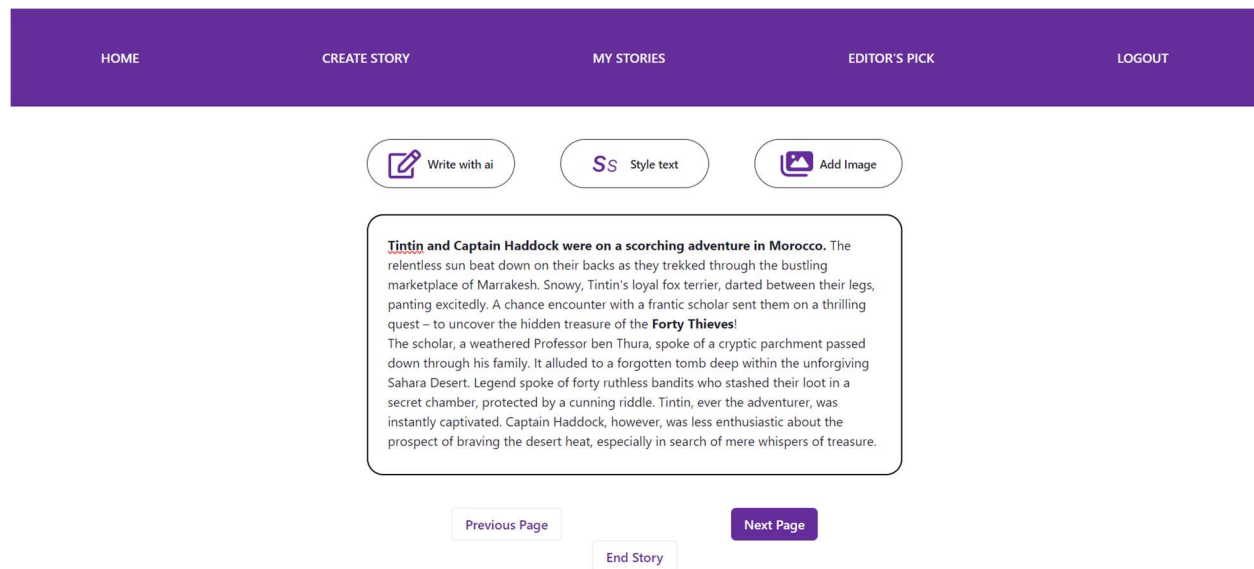


Figure 8 - Writing page

Story library Page

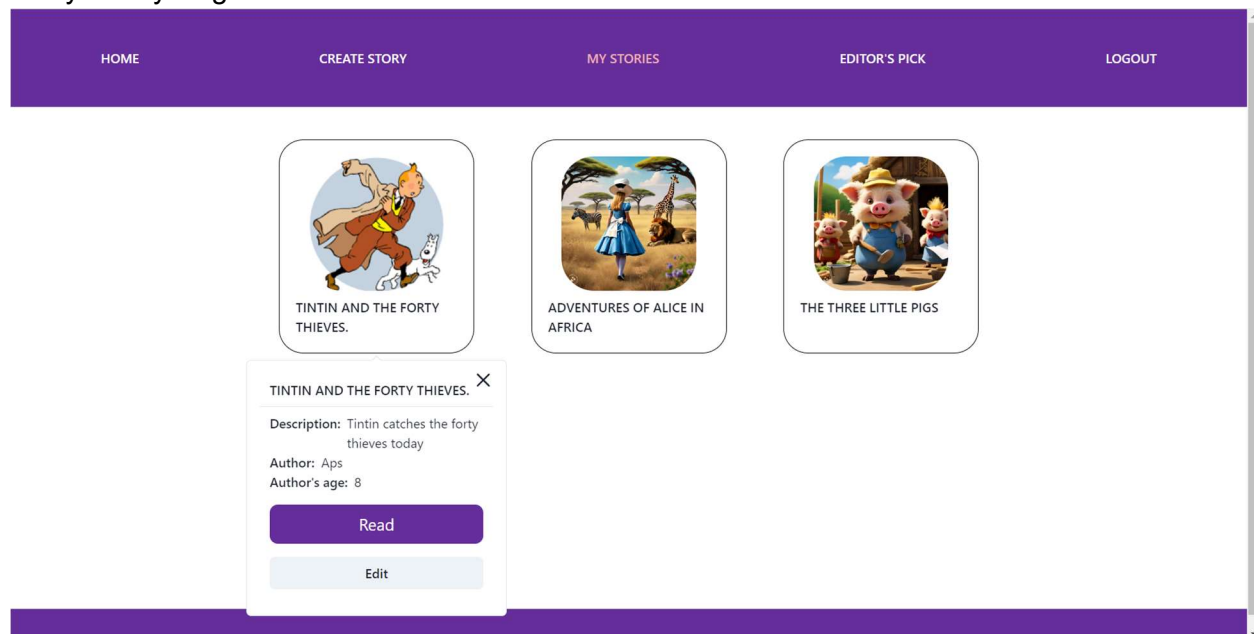


Figure 9- Library page

Reading Page

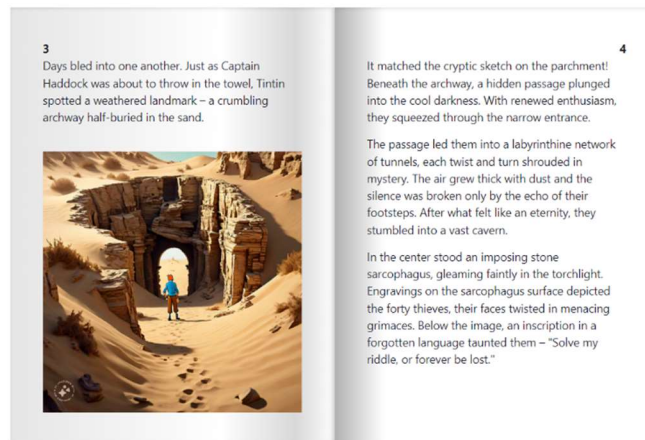
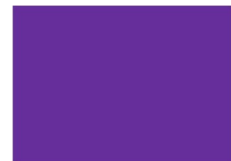


Figure 10 - Reading page

Rebeca Purple

Choosing the app's main color was crucial. With a target audience of children aged 3 to 11, we opted for a playful and memorable shade – Rebecca Purple. This vibrant color not only resonates with children but also holds a special meaning (online medium article).



Architecture

StoryNasi leverages a client-server architecture, separating the user interface(frontend) and the data storage and processing (backend) aspects. The frontend application requests data and functionality from the backend application using APIs.

Frontend Application

The frontend application consisted of three key areas: state management, pages, router.

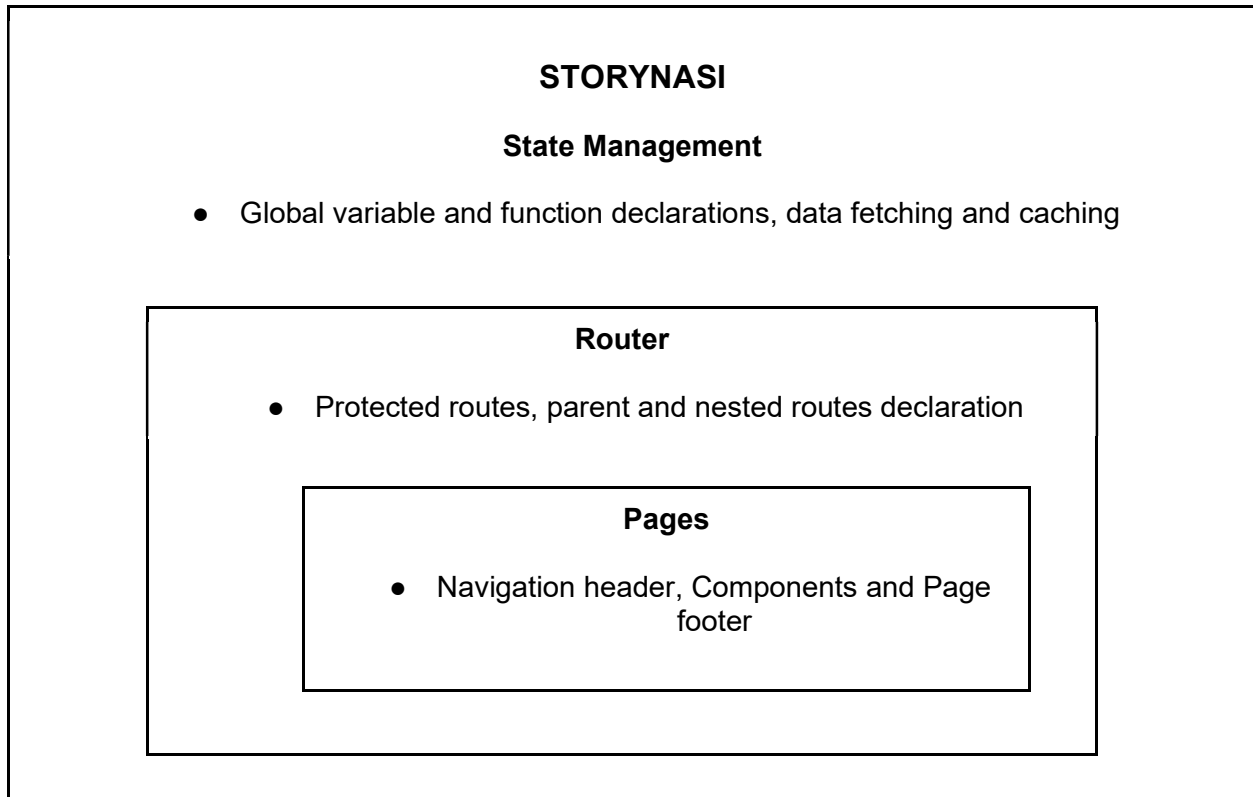


Figure 11 - Frontend core areas

State Management

We used ContextApi hook to encapsulate our application, enabling efficient state management and effortless propagation of variables and functions down the component tree. Also, the React useMemo hook and useCallback hook were used to cache data retrieved from API requests and functions, minimizing redundant rerenders and hence improving performance. Three contexts were created: AuthContext, CurrentStoryContext and LibraryContext.

The AuthContext manages user authentication, fetches user data from a backend API, restricts access to specific routes based on authentication status, and caches the result to avoid unnecessary rerenders.

The CurrentStoryContext manages the current story data (book information and pages) by offering functionalities to fetch book information, retrieve associated pages, and create or update pages.

The LibraryContext manages fetching all books from the database and caching the result to avoid redundant rerenders ensuring proper data management.

Router

The app router defines which components get rendered based on the current URL path. It also restricts access to specific routes for authenticated users.

Pages

An application page has three main sections, the header, body, and footer. The header component is only visible for authenticated users. The body contains one or more components. Some components are reused in other pages. One particularly interesting component is the EditorWithImage component. This component leverages Slate.js to provide an immersive and rich text editing experience.

Backend Application

The backend application can be divided into two main areas: data storage and routes.

Data Storage (database) • Config function • Models	Routes (end points) • Controllers • Middlewares
---	--

Figure 12 - Backend core areas

Data storage (database)

Our backend application interacts with the database in two ways: config and models. The Config function asynchronously attempts to connect to a MongoDB database using the Mongoose library. The models define the structure of data stored in the database. Each model represents a specific type of object in our code, specifying the fields and their data types.

Routes (endpoints)

Routes set up APIs that are used to manage data within an Express application. It utilizes functions from both controller and middleware. A controller specifies how data can be created, fetched, updated, and deleted pages. A middleware restricts who uses the data.

Architecture Overview

Figure (add number) shows the client architecture and figure (add number) shows how the client pages interact with the server side through API services.

Client

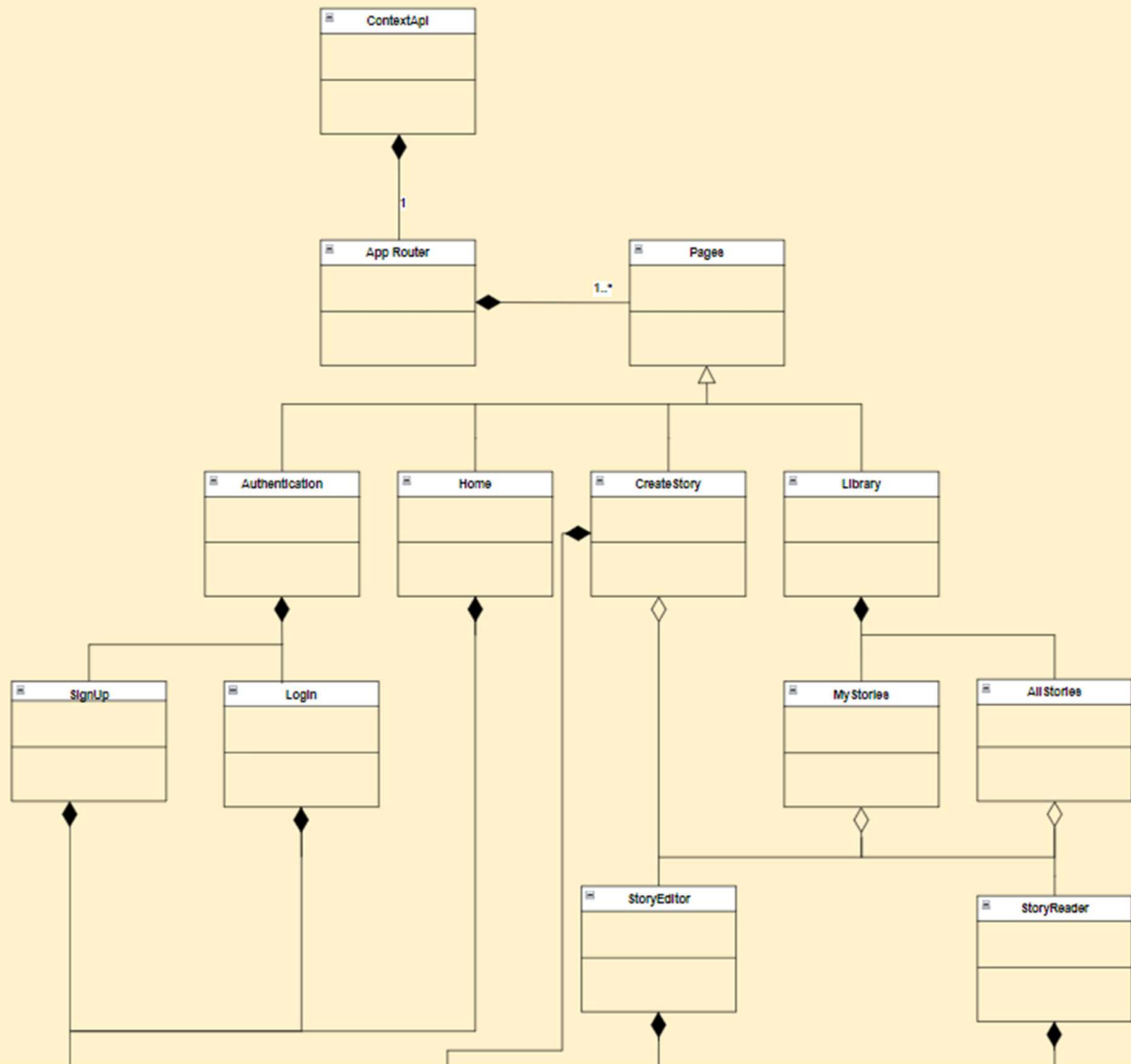


Figure 13 - Client architecture

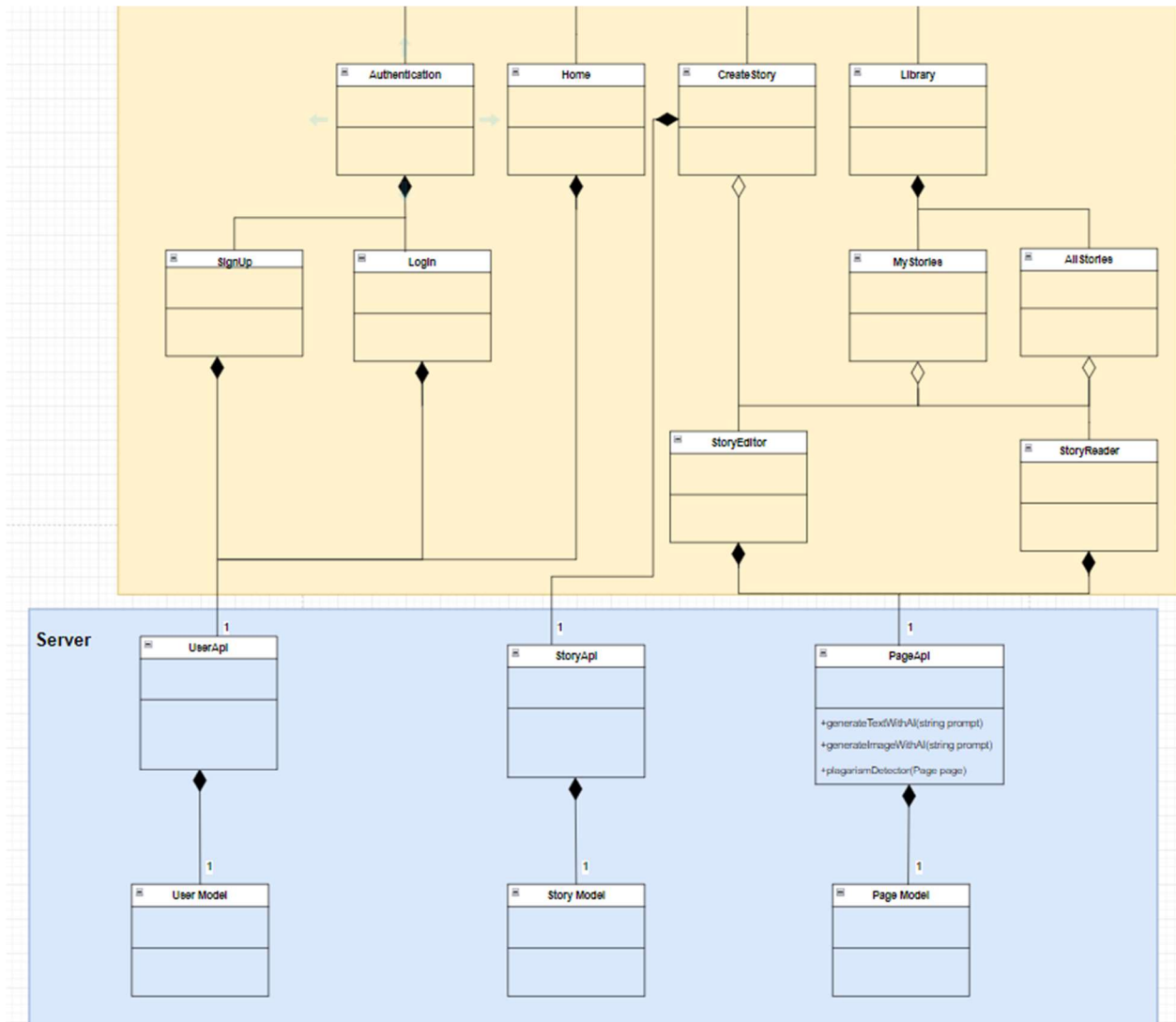


Figure 14 - User flow diagram

Implementation

This project is a web application designed for devices with a minimum width of 768 pixels. We used NPM, Javascript's package manager, for streamlined development and easier customization. All the packages used in the project are listed below.

Table 2 - NPM packages used in StoryNasi

Name	Purpose	Link
Axios	A promise-based HTTP client for making API requests.	https://github.com/axios

BcryptJS	A library for hashing passwords securely.	https://github.com/kelektiv/node.bcrypt.js/
Cors	Enables Cross-Origin Resource Sharing (CORS) for handling requests from different origins (domains, ports, protocols)	https://github.com/OpenLiberty/guide-cors
Dotenv	Loads environment variables from a .env file	https://github.com/dotenv-org
Express	A flexible web framework for building web applications and APIs. Download	https://www.npmjs.com/package/express
Jsonwebtoken	Creates, signs, and verifies JSON Web Tokens (JWTs) for secure authorization	https://www.npmjs.com/package/jsonwebtoken
Mongoose	Provides a simpler interface for interacting with MongoDB databases.	https://www.npmjs.com/package/mongoose
ChakraUI	Used for building accessible, responsive, and easy-to-use user interfaces	https://www.npmjs.com/package/@chakra-ui/react
Use-gesture	A library for handling touch and mouse input gestures in React applications	https://github.com/samselikoff/react-use-gesture
React	The core React library for building user interfaces and its DOM-specific counterpart for interacting with the browser's Document Object Model (DOM)	• https://react.dev/
React-icons	A collection of SVG icons for use in React projects.	https://www.npmjs.com/package/react-icons

React-router-dom	A routing library for building navigation experiences in React applications	https://www.npmjs.com/package/react-router-dom
React-spring	Animation library for creating book animation.	https://github.com/topics/react-spring
SlateJS	Customizable frameworks for building editors	https://docs.slatejs.org/

Results and Reflection

The project was fun to work on and at the end of the day we were happy with our application.

Release

StoryNasi's backend and frontend were successfully deployed on render.com on April 23, 2024. These are the links: <https://frontend-1-uip9.onrender.com/login>, <https://backend-ohox.onrender.com/api/>.

Reflection

Our initial vision for user-generated content involved a custom editor built within Unity, allowing for flexible placement of text and images. However, integrating the completed Unity WebGL build with our JavaScript code presented significant challenges. To overcome these interaction issues and ensure a smoother development process, we strategically shifted gears and adopted SlateJS, a powerful and customizable rich text editor library.

Future work

StoryNasi was a truly engaging and multifaceted project. Our team wore many hats, from brainstorming innovative features to designing the app, meticulously planning, implementing functionalities, and finally deploying it.

While there's always room for growth, StoryNasi boasts a solid foundation. Some areas for future development include integrating features like responsive design for smaller screens (max width 768px) and precise text and image positioning within the book. Additionally, smaller enhancements are in the works, such as user-uploaded images and color customization for book backgrounds.

Overall, working on StoryNasi has been an enriching and rewarding experience.

References

Credit

I'd like to express my sincere thanks to Jonathan Engelsma for his incredible mentorship. His guidance was crucial in helping me navigate challenges and communicate effectively with the university. I particularly appreciated his support when I got stuck integrating the Unity WebGL build with Javascript. His insights helped me choose the right editor and move forward.

Tools

Here is a list of some of the technical stack we used for creating Story nasi.

Table 3 - Tools used to create app

Name	Url
Figma	https://www.figma.com/
NPM	https://www.npmjs.com/
MongoDB	https://www.mongodb.com/
Diagrams.net	https://app.diagrams.net/
Dbdiagram	https://dbdiagram.io/home
Unity	https://unity.com/
Render	https://render.com/

Figures

Figure 1 - Structure of the book publishing industry (Vanier, 1973 p.5).....	5
Figure 2 - User flow diagram	8
Figure 3 - Database schema.....	9
Figure 4 - Signup page.....	10
Figure 5 - Login page.....	10
Figure 6 - Home page.....	11
Figure 7 - Create story page.....	11
Figure 8 - Writing page.....	12
Figure 9 - Library page.....	12
Figure 10 - Reading page.....	13

Figure 11 - Frontend core areas.....	14
Figure 12 - Backend core areas.....	15
Figure 13 - Client architecture.....	16
Figure 14 - Client server interaction.....	17

Tables

Table 1 - Application features.....	6
Table 2 - NPM packages used in StoryNasi.....	17
Table 3 - Tools used to create web app.....	20

Bibliography

1. Vanier, D.J. Market Structure and the Business of Book Publishing. Pitman Publishing Company, New York, (1973). [Cited: April 21, 2024.]
2. Algaze, V. The “Hidden” Purple Memorial in Your Web Browser. April 5, 2017. [Cited: April 21, 2024.] <https://medium.com/@valgaze/the-hidden-purple-memorial-in-your-web-browser-7d84813bb416>.
3. Slate. Slate. [Online] Slate, 2022. [Cited: April 29, 2024.] <https://docs.slatejs.org/>.
4. React spring. Use spring. [Online] React Spring, 2024. [Cited: April 29, 2024.] <https://www.react-spring.dev/docs/components/use-spring>.
5. NPM. @use-gesture. [Online] NPM, 2024. [Cited: April 29, 2024.] <https://www.npmjs.com/package/@use-gesture/react>